

CSA1718 AI
LAB PROBLEMS

SANJAI.S(192511018)

EXPERIMENT 11:

TIC Tac Toe Game

AIM

To develop a Tic Tac Toe game in Python where two players play alternately until one player wins or the game ends in a draw.

ALGORITHM

1. Start.
2. Create an empty 3×3 game board.
3. Set the first player as 'X'.
4. Display the board.
5. Ask the current player to enter a position.
6. If the position is empty, place the player's symbol.
7. Check whether the player has won.
8. If a player wins, display the winner and stop.
9. Otherwise, switch to the other player.
10. Repeat until all 9 positions are filled.
11. If no player wins, declare the game as a draw.
12. Stop.

PSEUDOCODE

START

Create empty board

Set current player = X

Repeat 9 times

Display board

Read player position

If position is empty

Place player's symbol

Else

Ask for another position

Check winning combinations

If player wins

Display winner

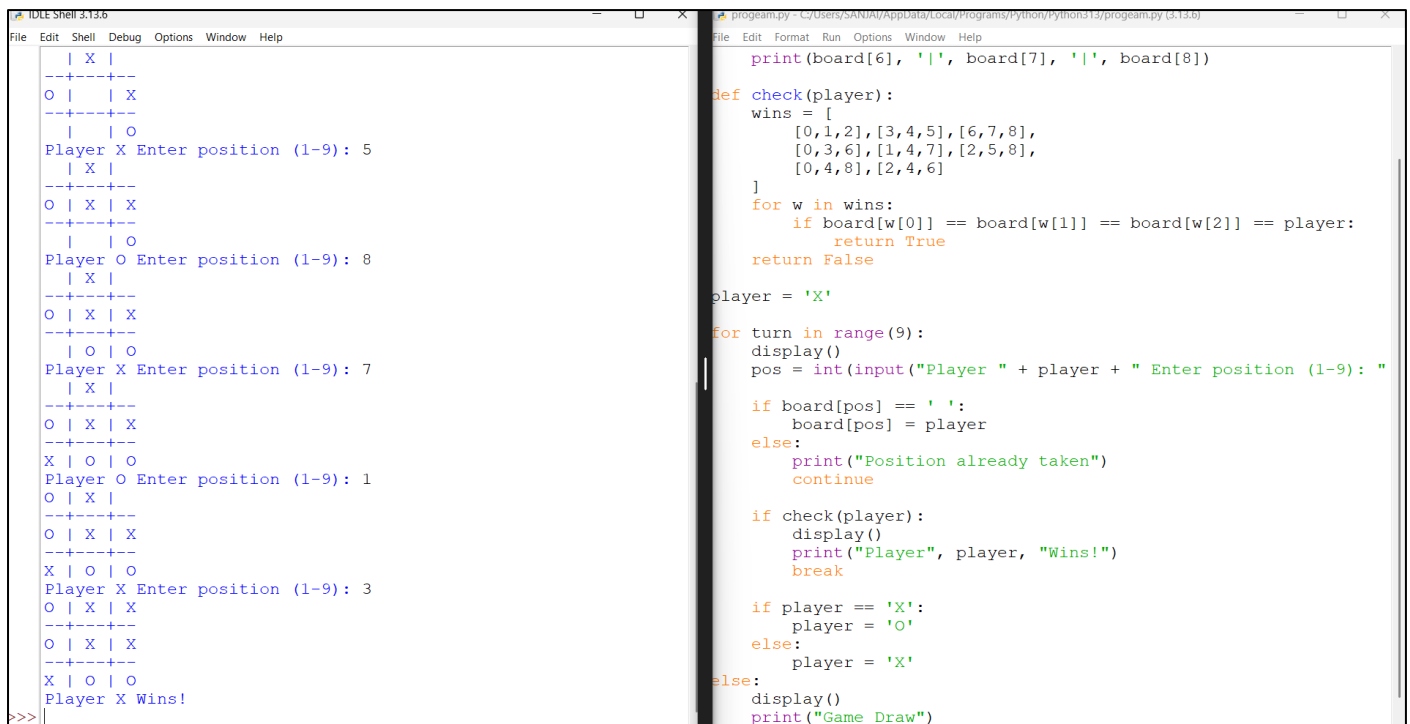
STOP

Change player

Display "Game Draw"

STOP

OUTPUT:



```

TIDLE SHELL 3.13.6
File Edit Shell Debug Options Window Help
| X |
|---|
O | | X
|---|
| | O
Player X Enter position (1-9): 5
| X |
|---|
O | X | X
|---|
| | O
Player O Enter position (1-9): 8
| X |
|---|
O | X | X
|---|
| O | O
Player X Enter position (1-9): 7
| X |
|---|
O | X | X
|---|
X | O | O
Player O Enter position (1-9): 1
O | X |
|---|
O | X | X
|---|
X | O | O
Player X Enter position (1-9): 3
O | X | X
|---|
O | X | X
|---|
X | O | O
Player X Wins!
>>>

progeam.py - C:/Users/SANIA/AppData/Local/Programs/Python/Python313/progeam.py (3.13.6)
File Edit Format Run Options Window Help
print(board[6], '|', board[7], '|', board[8])

def check(player):
    wins = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for w in wins:
        if board[w[0]] == board[w[1]] == board[w[2]] == player:
            return True
    return False

player = 'X'

for turn in range(9):
    display()
    pos = int(input("Player " + player + " Enter position (1-9): "))

    if board[pos] == ' ':
        board[pos] = player
    else:
        print("Position already taken")
        continue

    if check(player):
        display()
        print("Player", player, "Wins!")
        break

    if player == 'X':
        player = 'O'
    else:
        player = 'X'

else:
    display()
    print("Game Draw")
```

Result

The Tic Tac Toe game was successfully implemented in Python. The program allows two players to play alternately, correctly detects the winner, and declares a draw when no winning condition is achieved.

These are concise, lab-record-friendly Aim, Algorithm, Pseudocode, and Result sections suitable for practical examinations.

EXPERIMENT 12:

MIN MAX ALGO

AIM

To implement the Minimax Algorithm for the Tic Tac Toe game, enabling the computer to always choose the optimal move against the human player.

ALGORITHM

1. Start.
2. Create a 3×3 Tic Tac Toe board.
3. Allow the human player to make a move.
4. Check if the game has reached a terminal state (win, lose, or draw).
5. If it is the computer's turn:
 - Generate all possible moves.
 - Apply the Minimax algorithm to each move.
 - If it is the computer's turn (MAX), choose the move with the highest score.
 - If it is the human's turn (MIN), choose the move with the lowest score.
6. Place the best move on the board.
7. Repeat until a player wins or the game ends in a draw.
8. Display the result.
9. Stop.

PSEUDOCODE

START

Initialize board

FUNCTION Minimax(board, isMax)

IF computer wins

RETURN +1

IF human wins

RETURN -1

IF draw

RETURN 0

IF isMax

best = $-\infty$

FOR each empty cell

Place computer move

score = Minimax(board, False)

Remove move

best = max(best, score)

RETURN best

ELSE

best = $+\infty$

FOR each empty cell

Place human move

score = Minimax(board, True)

Remove move

best = min(best, score)

RETURN best

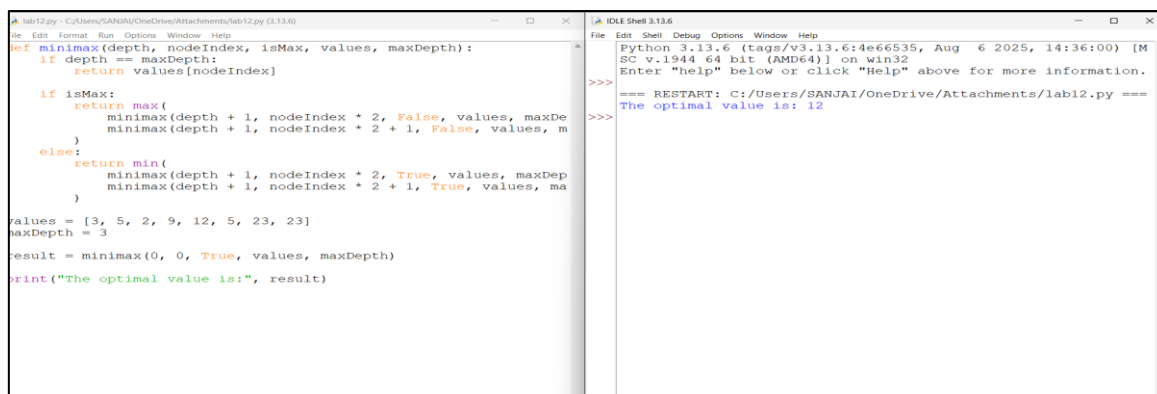
END FUNCTION

Computer selects move with highest Minimax score.

Display Winner or Draw.

STOP

OUTPUT



The screenshot shows two windows. The left window is a Python IDE with the following code:

```
def minimax(depth, nodeIndex, isMax, values, maxDepth):  
    if depth == maxDepth:  
        return values[nodeIndex]  
  
    if isMax:  
        return max(  
            minimax(depth + 1, nodeIndex * 2, False, values, maxDepth),  
            minimax(depth + 1, nodeIndex * 2 + 1, False, values, maxDepth)  
        )  
    else:  
        return min(  
            minimax(depth + 1, nodeIndex * 2, True, values, maxDepth),  
            minimax(depth + 1, nodeIndex * 2 + 1, True, values, maxDepth)  
        )  
  
values = [3, 5, 2, 9, 12, 5, 23, 23]  
maxDepth = 3  
result = minimax(0, 0, True, values, maxDepth)  
print("The optimal value is:", result)
```

The right window is a command prompt showing the output of the program:

```
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [MSC v.1944 64 bit (AMD64)] on win32  
>>> Enter "help" below or click "Help" above for more information.  
=== RESTART: C:/Users/SANJAI/OneDrive/Attachments/lab12.py ===  
>>> The optimal value is: 12  
>>>
```

RESULT

The Tic Tac Toe game was successfully implemented using the Minimax Algorithm. The computer evaluates all possible future game states and always selects the optimal move, ensuring the best possible outcome (win or draw).

EXPERIMENT 13:

ALPHA BETA ALGORITHM

AIM

To implement the Alpha-Beta Pruning algorithm to optimize the Minimax search by eliminating branches that cannot affect the final decision and determine the optimal move in a game tree.

ALGORITHM

1. Start at the root node with:
 - Alpha (α) = $-\infty$
 - Beta (β) = $+\infty$
2. If the current node is a terminal node, return its utility value.
3. If the current node is a MAX node:
 - Initialize the best value as $-\infty$.
 - Evaluate each child recursively.
 - Update the best value.
 - Update $\alpha = \max(\alpha, \text{best value})$.
 - If $\alpha \geq \beta$, stop exploring the remaining children (Beta Cutoff).
4. If the current node is a MIN node:
 - Initialize the best value as $+\infty$.
 - Evaluate each child recursively.
 - Update the best value.
 - Update $\beta = \min(\beta, \text{best value})$.
 - If $\beta \leq \alpha$, stop exploring the remaining children (Alpha Cutoff).
5. Return the best value obtained.
6. Continue until the root node returns the optimal move.

PSEUDOCODE

FUNCTION AlphaBeta(node, depth, alpha, beta, maximizingPlayer)

IF node is terminal OR depth = 0 THEN

 RETURN node.value

END IF

IF maximizingPlayer THEN

 value $\leftarrow -\infty$

FOR each child of node DO

value $\leftarrow$ **MAX**(value,

AlphaBeta(child, depth-1, alpha, beta, FALSE))

alpha $\leftarrow$ **MAX**(alpha, value)

IF $\alpha \geq \beta$ **THEN**

BREAK

END IF

END FOR

RETURN value

ELSE

value $\leftarrow +\infty$

FOR each child of node DO

value $\leftarrow$ **MIN**(value,

AlphaBeta(child, depth-1, alpha, beta, TRUE))

beta $\leftarrow$ **MIN**(beta, value)

IF $\beta \leq \alpha$ **THEN**

BREAK

END IF

END FOR

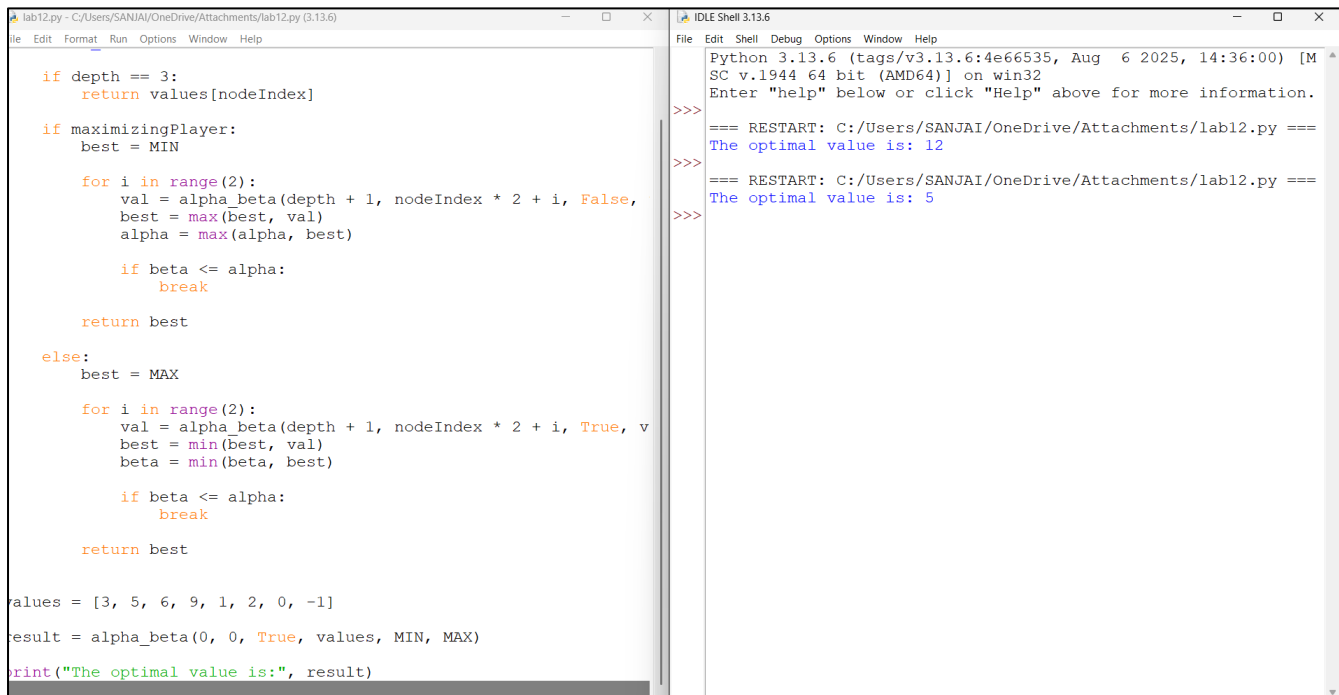
RETURN value

END IF

END FUNCTION

This is the standard Aim, Algorithm, and Pseudocode format commonly accepted for AI lab records.

OUTPUT:



The image shows a screenshot of a Python IDE with two windows. The left window, titled 'lab12.py', contains the following Python code:

```
def alpha_beta(depth, nodeIndex, maximizingPlayer, values, MIN, MAX):
    if depth == 3:
        return values[nodeIndex]

    if maximizingPlayer:
        best = MIN

        for i in range(2):
            val = alpha_beta(depth + 1, nodeIndex * 2 + i, False, values, MIN, MAX)
            best = max(best, val)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best
    else:
        best = MAX

        for i in range(2):
            val = alpha_beta(depth + 1, nodeIndex * 2 + i, True, values, MIN, MAX)
            best = min(best, val)
            beta = min(beta, best)

            if beta <= alpha:
                break

        return best

values = [3, 5, 6, 9, 1, 2, 0, -1]
result = alpha_beta(0, 0, True, values, MIN, MAX)
print("The optimal value is:", result)
```

The right window, titled 'IDLE Shell 3.13.6', shows the output of the program:

```
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/SANJAI/OneDrive/Attachments/lab12.py ===
The optimal value is: 12
>>>
=== RESTART: C:/Users/SANJAI/OneDrive/Attachments/lab12.py ===
The optimal value is: 5
>>>
```

Result:

The Alpha-Beta Pruning algorithm successfully determined the optimal move with a utility value of 5, while reducing the number of nodes evaluated through pruning compared to the standard Minimax algorithm.